KPR Institute of Engineering and Technology

Department of Artificial Intelligence and Data Science

PROJECT REPORT

Exploratory Data Analysis and Visualization of the Breast Cancer Wisconsin (Diagnostic) Dataset with a Deep Learning Classifier

Course Code: U21ADP05
Course Title: Exploratory Data Analysis and Visualization
Assignment II – Case Study (Academic Year 2026–2027, Odd Semester)

Submitted by

Name: Savitha T
Roll No: 24AD107
Department:Artifcial Intelligence and Data Science
Year/Semeter: 3rd Year/05

1.Abstract

Accurate classification of breast masses as benign or malignant supports timely clinical attention. This project performs exploratory data analysis (EDA) and visualization of the Breast Cancer Wisconsin (Diagnostic) dataset, containing 569 records with 30 numerical features computed from digitized images of fine needle aspirates; 357 are benign and 212 malignant. The data were checked for missing values, duplicates and outliers, standardized, and split into 70% training, 15% validation and 15% test sets. Histograms, boxplots, a correlation heatmap, scatter plot, pairplot and class-wise comparisons revealed that size and shape features separate the classes strongly. A multilayer perceptron was trained and tuned using validation AUC. On the 86-sample test set it achieved 98.84% accuracy, 100% precision, 96.88% recall, an F1-score of 0.9841 and ROC-AUC of 1.0000. The study is educational and not a diagnostic tool.

2. Introduction and Objective

Breast cancer is one of the most common cancers worldwide, and early detection greatly improves the chances of successful treatment. One way to examine a breast lump is fine needle aspiration (FNA), in which a small sample of cells is extracted and viewed under a microscope. The Breast Cancer Wisconsin (Diagnostic) dataset describes the cell nuclei seen in digitized FNA images using numerical measurements such as radius, texture, perimeter, area and concavity, together with a label stating whether the mass was benign or malignant.

Exploratory Data Analysis (EDA) is the process of examining a dataset before modelling in order to understand its structure, quality, distributions and relationships. Visualization makes these properties visible, so that patterns can be recognised and data-quality problems can be detected. This project applies EDA and visualization to the dataset and then trains a deep learning model, a Multilayer Perceptron (MLP), to classify masses as benign or malignant.

2.1 Objectives

- To load, inspect and clean the dataset (missing values, duplicates, outliers, unnecessary columns).
- To encode the target label, standardize the features and split the data into training, validation and test sets.
- To create at least five meaningful visualizations and interpret what each reveals about the data.
- To build, tune and evaluate an MLP classifier and visualize its performance (loss and accuracy curves, confusion matrix, ROC curve and AUC).
- To compare the MLP with a Logistic Regression baseline and identify the features that influence the model most.

Important note: this project is an academic exercise. The model was built on a small, well-known research dataset and has not been clinically validated. It is not intended to diagnose real patients or to replace the judgement of medical professionals.

3. Dataset Description

3.1 Source

The dataset is the Breast Cancer Wisconsin (Diagnostic) dataset, downloaded from Kaggle (uciml/breast-cancer-wisconsin-data): https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data. It is a copy of the dataset in the UCI Machine Learning Repository, which was

created by W. H. Wolberg, W. N. Street and O. L. Mangasarian at the University of Wisconsin [1], [2].

3.2 Size and Fields

The raw CSV file has **569 rows and 33 columns**. The columns are an identifier (id), the target (diagnosis), 30 numerical feature columns, and one column named "Unnamed: 32" that contains no values at all (0 non-null entries). The identifier and the empty column were removed, leaving **569 records, 30 numerical features and one target variable**. This satisfies the assignment requirement of at least 15 features.

The 30 features are built from ten measurements of each cell nucleus. For each measurement, three values are provided: the **mean**, the **standard error (se)** and the **worst** value (the mean of the three largest values), as shown in Table 1.

Base measurement	Mean feature	Standard-error feature	Worst feature
Radius	radius_mean	radius_se	radius_worst
Texture	texture_mean	texture_se	texture_worst
Perimeter	perimeter_mean	perimeter_se	perimeter_worst
Area	area_mean	area_se	area_worst
Smoothness	smoothness_mean	smoothness_se	smoothness_worst
Compactness	compactness_mean	compactness_se	compactness_worst
Concavity	concavity_mean	concavity_se	concavity_worst
Concave points	concave points_mean	concave points_se	concave points_worst
Symmetry	symmetry_mean	symmetry_se	symmetry_worst
Fractal dimension	fractal_dimension_mean	fractal_dimension_se	fractal_dimension_worst

Table 1: The 30 numerical features (10 measurements × 3 statistics)

3.3 Target Classes

The target column "diagnosis" has two classes: **B = Benign** and **M = Malignant**. In the notebook they were encoded as 0 and 1 respectively.

Class	Label	Encoded as	Records	Percentage
Benign	B	0	357	62.74%
Malignant	M	1	212	37.26%

Table 2: Class distribution (569 records). The classes are moderately imbalanced.

3.4 Basic Statistics

Table 3 shows the mean, standard deviation and minimum of selected features, taken from the statistical summary in the notebook (df.describe()). The features lie on very different scales: for example, area_mean averages about 655, whereas smoothness_mean averages about 0.096. This is the main reason why feature scaling is needed before training a neural network.

Feature	Mean	Std. deviation	Minimum
radius_mean	14.127	3.524	6.981

Feature	Mean	Std. deviation	Minimum
texture_mean	19.290	4.301	9.710
perimeter_mean	91.969	24.299	43.790
area_mean	654.889	351.914	143.500
smoothness_mean	0.0964	0.0141	0.0526
concavity_mean	0.0888	0.0797	0.0000
area_worst	880.583	569.357	185.200
concavity_worst	0.2722	0.2086	0.0000

Table 3: Summary statistics of selected features (all 569 records)

The notebook also compared feature means for the two classes. Table 4 lists some of them. Malignant cases have clearly larger values for size- and shape-related features.

Feature	Benign mean	Malignant mean
radius_mean	12.147	17.463
area_mean	462.790	978.376
concavity_mean	0.0461	0.1608
concave points_mean	0.0257	0.0880
area_worst	558.899	1422.287
concave points_worst	0.0744	0.1822
fractal_dimension_mean	0.0629	0.0627

Table 4: Class-wise mean values of selected features

4. EDA and Preprocessing

4.1 Loading and Inspection

The CSV file was loaded with pandas. The shape (569, 33), the first five rows, the column names, the data types and the non-null counts were printed, followed by df.describe(). All feature columns are numeric (float64), the id column is an integer, and diagnosis is a text (object) column.

4.2 Missing-Value Handling

Missing values were checked column by column. **None of the 30 feature columns or the diagnosis column contains missing values.** The only empty column was “Unnamed: 32”, which has 0 non-null entries; it is an artefact of the CSV file and was dropped together with the id column. The notebook also contains a median imputer (SimpleImputer) fitted on the training data only. Because the number of missing feature values was 0, the imputer did not change any value; it acts only as a safeguard that would handle missing values if the pipeline were reused on other data.

4.3 Duplicate Checking

Duplicate rows were checked after removing the id column. The count was **0**, so no records were removed.

4.4 Outlier Analysis

Outliers were analysed using the Interquartile Range (IQR) rule: a value is flagged if it is below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR. Boxplots were drawn for four selected features, and flagged values were counted for every feature. **29 of the 30 features contained at least one flagged value**; only concave points_worst had none. Table 5 shows selected counts.

Feature	Flagged values	Feature	Flagged values
area_se	65	area_mean	25
radius_se	38	radius_mean	14
perimeter_se	38	perimeter_mean	13
area_worst	35	texture_mean	7
smoothness_se	30	texture_worst	5

Table 5: IQR-based potential outliers in selected features (out of 569 records)

Treatment decision: the flagged values were **not removed or capped** in the notebook. The dataset is small (569 records), and extreme values in size-related features may be genuine measurements of large nuclei rather than errors, so deleting them could remove real information. The values were therefore kept and handled by standardization. The IQR rule only flags values that are unusual compared with the rest of the data; it does not prove that they are errors.

4.5 Encoding

The only categorical column is the target, diagnosis. It was label-encoded as **B → 0 (Benign)** and **M → 1 (Malignant)**. All 30 features are already numeric, so no one-hot encoding was needed. Before the split, the notebook also verified that all target values were valid (no unmapped labels).

4.6 Feature Scaling

The features were **standardized** with StandardScaler, which transforms each feature to have zero mean and unit variance ($z = (x - \mu) / \sigma$). This prevents large-valued features such as area from dominating small-valued ones such as smoothness during neural-network training. The scaler learned μ and σ **from the training set only** and then applied the same transformation to the validation and test sets. This avoids data leakage, which would otherwise make the test results look better than they truly are.

4.7 Train / Validation / Test Split

The data were split with stratification, so that the benign/malignant proportions remain similar in every subset, using random_state = 42: first 70% for training and 30% held out, then the held-out part was divided equally into validation and test sets.

Subset	Records	Share	Purpose
Training set	398	≈ 70%	Fit the model weights
Validation set	85	≈ 15%	Early stopping and choosing the hyperparameters
Test set	86	≈ 15%	Final, unbiased evaluation (54 benign, 32 malignant)

Table 6: Dataset split (each subset has 30 features)

4.8 Important EDA Findings

- The data are clean: no missing values, no duplicates; only an id column and one empty column had to be removed.
- The classes are moderately imbalanced (62.74% benign, 37.26% malignant), which is why stratified splitting was used and why precision, recall and F1-score are reported in addition to accuracy.
- The features have very different scales, so standardization is required.
- The selected size features are right-skewed (skewness: radius_mean 0.94, texture_mean 0.65, perimeter_mean 0.99, area_mean 1.65).
- Radius, perimeter and area are very strongly correlated with one another, so many features carry overlapping information (multicollinearity).
- Malignant cases have noticeably larger values for size, concavity and concave-points features than benign cases.

5. Data Visualization

The visualizations were created in Python using Matplotlib and Seaborn. Seven visualizations from the notebook are explained below; the assignment requires at least five. The figures are the actual plots produced by the executed notebook.

5.1 The Seven Stages of Visualization

The assignment asks that the visualization work cover the seven stages of the visualization process (acquire, parse, filter, mine, represent, refine and interact) [8]. Table 7 shows how each stage was handled in this project.

Stage	What was done in this project	Status
Acquire	Dataset downloaded from Kaggle and loaded from CSV with pandas.	Done
Parse	Column types checked; the target was encoded as 0/1; the id and empty columns identified.	Done
Filter	Removed id and Unnamed: 32; selected four size features for the histogram, boxplot and pairplot.	Done
Mine	Skewness, IQR outlier counts, correlation matrix and class-wise means.	Done
Represent	Count plot, histograms, boxplot, heatmap, scatter plot, pairplot and bar chart.	Done
Refine	Titles, axis labels, legends, colours by class and a clean Seaborn theme were applied.	Done (partly; see limitations)
Interact	Interactive charts (hover, zoom, filter) were not created in the notebook.	Not done: see Section 9 (future scope)

Table 7: The seven stages of visualization and their status in this project

5.2 Visualizations

Visualization 1: Class Distribution (Count Plot)

Purpose: To check how many benign and malignant cases the dataset contains, because class imbalance affects model training and the choice of evaluation metrics.

What it shows: A bar chart of the number of records in each diagnosis class.

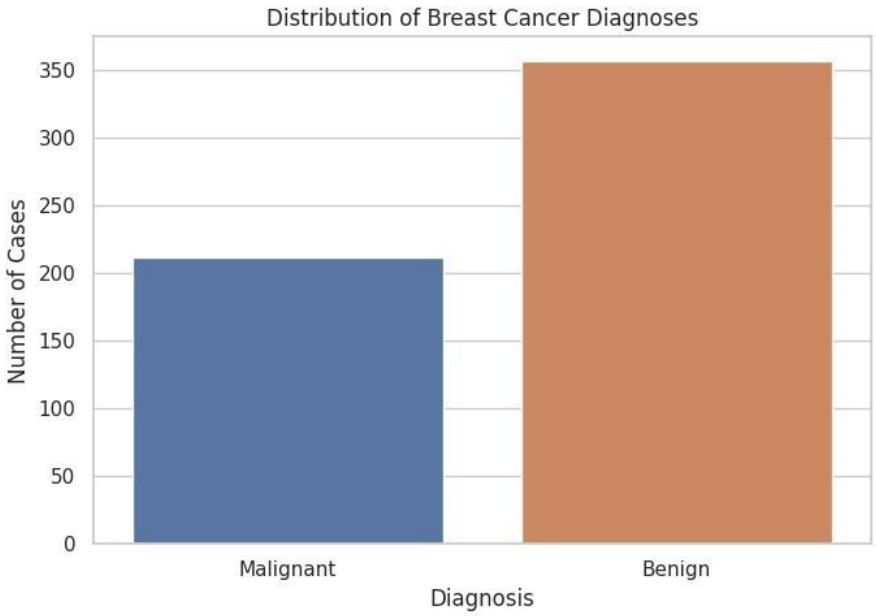


Figure 1: Distribution of Breast Cancer Diagnoses

Key observations: There are 357 benign and 212 malignant cases (62.74% and 37.26%). Benign cases are about 1.7 times as frequent as malignant ones.

Interpretation: The imbalance is moderate, not severe, so the model can still learn both classes. It does, however, mean that accuracy alone could look good even if the minority (malignant) class were handled poorly. Stratified splitting and recall/precision/F1 were therefore used.

Visualization 2: Histograms of Selected Features (Univariate Analysis)

Purpose: To understand the shape of the distribution of individual features (centre, spread and skewness).

What it shows: Histograms with 20 bins for radius_mean, texture_mean, perimeter_mean and area_mean. The skewness of each feature was also calculated.

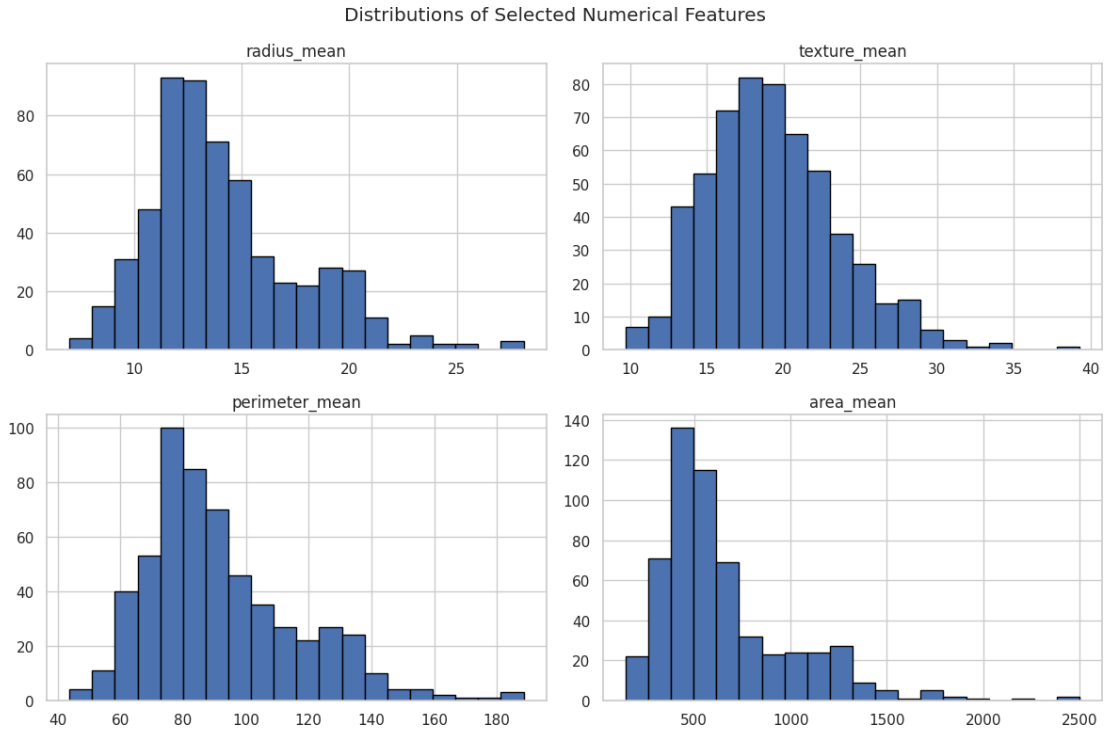


Figure 2: Distributions of Selected Numerical Features

Key observations: All four features are right-skewed, with a long tail towards large values. The skewness values are 0.94 (radius_mean), 0.65 (texture_mean), 0.99 (perimeter_mean) and 1.65 (area_mean). Most radius values are around 12–15, and most area values are below about 1000. texture_mean is the closest to a symmetric bell shape.

Interpretation: The right tail means that a minority of tumours have much larger cells than the rest. This is consistent with the dataset mixing two populations (benign and malignant), and it explains why standardization is useful. The strongest skew is in area_mean, since area grows with the square of the radius.

Visualization 3: Boxplots of Selected Features (Outlier Analysis)

Purpose: To detect extreme values and to compare the spread of the features.

What it shows: Horizontal boxplots of radius_mean, texture_mean, perimeter_mean and area_mean. The IQR rule flagged 14, 7, 13 and 25 values respectively.

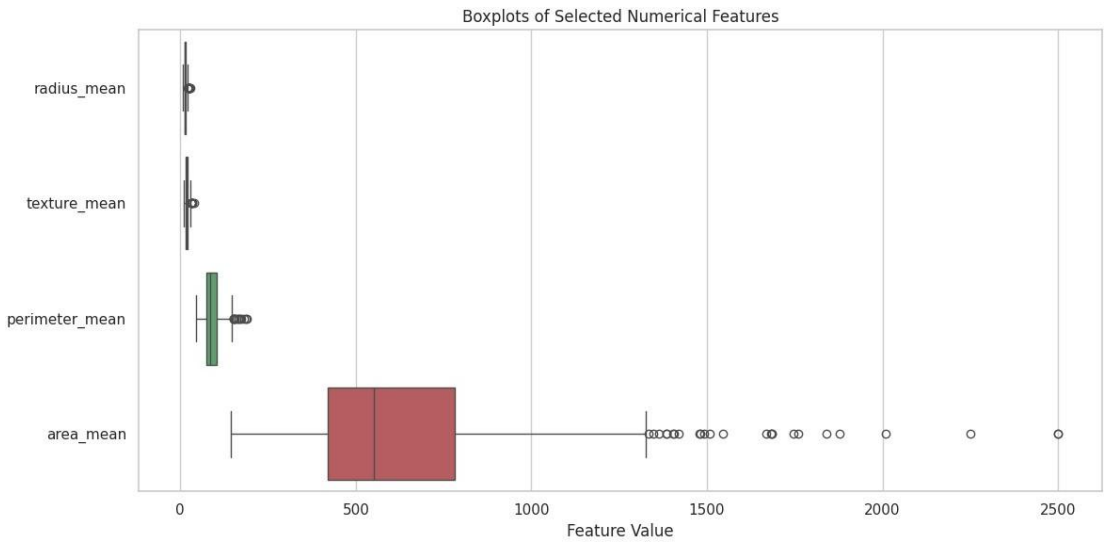


Figure 3: Boxplots of Selected Numerical Features

Key observations: area_mean has the widest box and the largest number of high-value points (several above 1500, with a maximum near 2500). Because the features are plotted on one shared axis without scaling, the boxes of radius_mean and texture_mean look very small and are hard to read.

Interpretation: The outliers lie on the upper side only, which matches the right-skewed histograms. The unequal scales also show again why standardization is needed. The flagged values were kept (see Section 5.4). A clearer version of this plot would standardize the features first or use a separate axis for each feature.

Visualization 4: Correlation Heatmap (Multivariate Analysis)

Purpose: To find which features are related to each other and to detect redundant features.

What it shows: A colour-coded matrix of Pearson correlation coefficients between all 30 features (red = positive, blue = negative, white/grey = near zero).

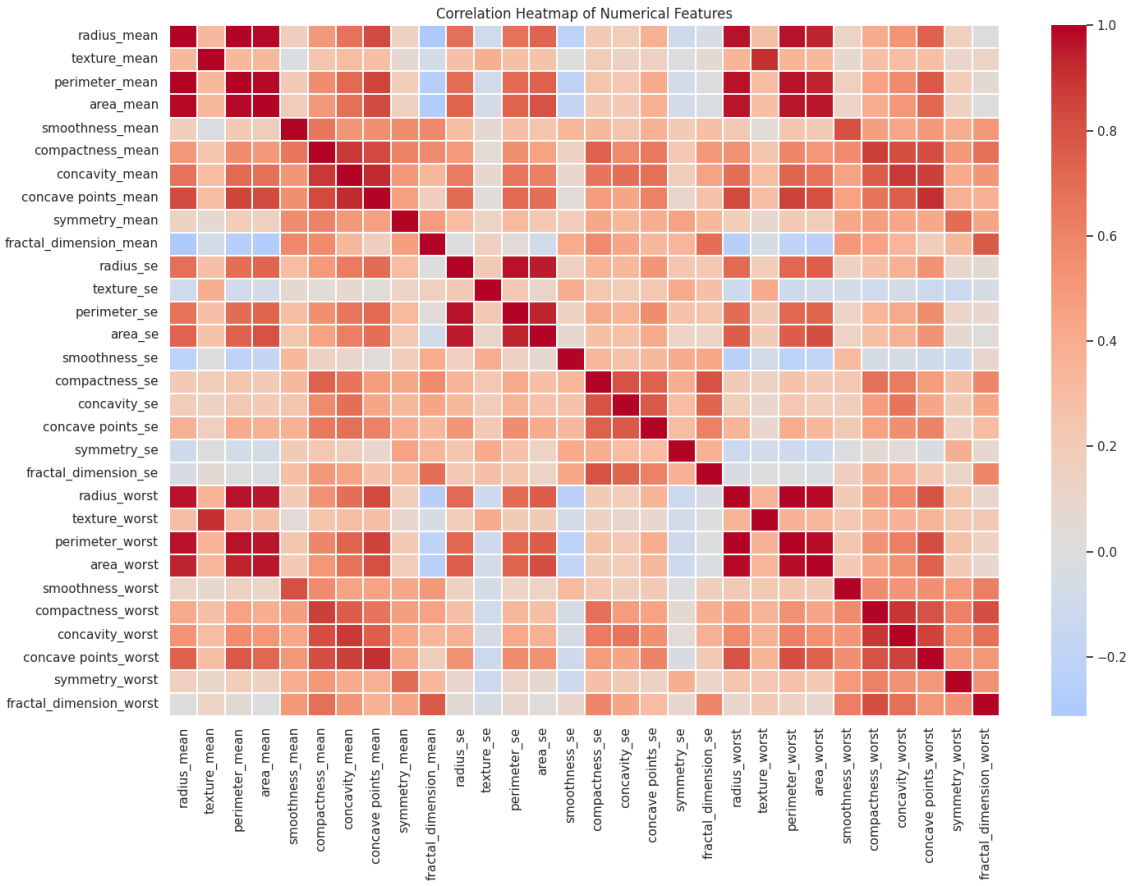


Figure 4: Correlation Heatmap of Numerical Features

Key observations: There is a bright-red block between radius, perimeter and area (mean and worst versions), meaning they are almost perfectly positively correlated. Another group of strongly positive cells is formed by compactness, concavity and concave points. A few weak negative (blue) cells appear, for example for fractal_dimension_mean with the size features.

Interpretation: Many features carry overlapping information, so the 30 features are not 30 independent signals. For a neural network this is not a problem for prediction, but it makes it hard to say which one of a group of correlated features is "responsible" for the result, which affects the feature-importance analysis in Section 8. For linear models, such multicollinearity can make the coefficients unstable.

Visualization 5: Scatter Plot: Mean Radius vs Mean Texture by Diagnosis

Purpose: To see whether two features can separate benign from malignant cases and to study the relationship between them.

What it shows: Each point is one patient record, with radius_mean on the x-axis, texture_mean on the y-axis, and colour showing the diagnosis.

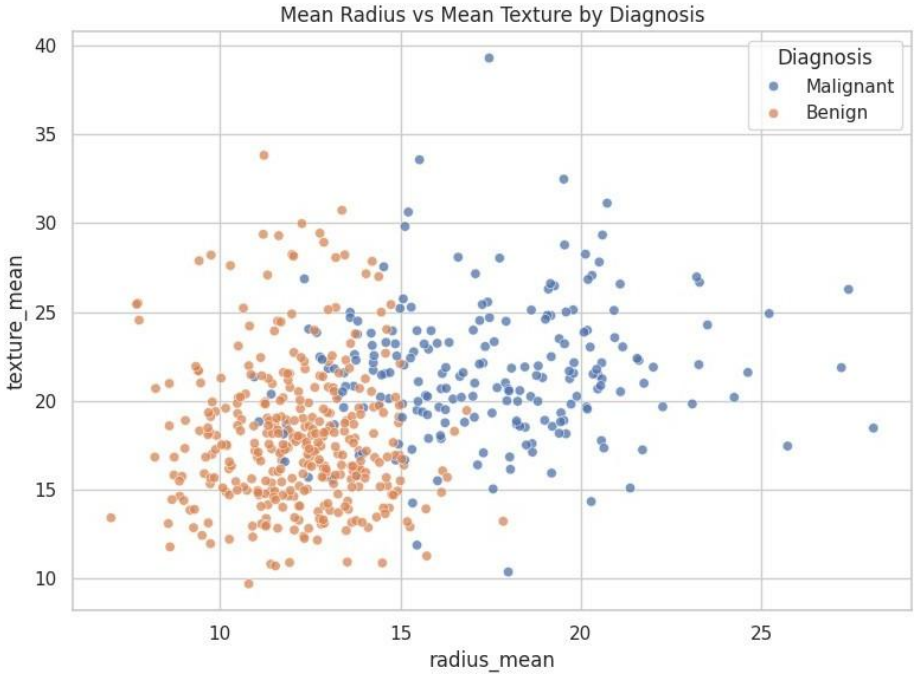


Figure 5: Mean Radius vs Mean Texture by Diagnosis

Key observations: Benign points (orange) are concentrated at smaller radius values, roughly 8–15, whereas malignant points (blue) are spread over larger radius values, mostly above about 14. The two groups overlap in the middle region and along the texture axis.

Interpretation: radius_mean is a stronger separator than texture_mean, since the classes are mostly separated left-to-right. A single pair of features does not separate the classes completely, which motivates the use of all 30 features in a model.

Visualization 6: Pairplot of Size Features Coloured by Diagnosis

Purpose: To explore several pairwise relationships and class-wise distributions in one view.

What it shows: A lower-triangle matrix of scatter plots for radius_mean, texture_mean, perimeter_mean and area_mean, with class-coloured histograms on the diagonal.

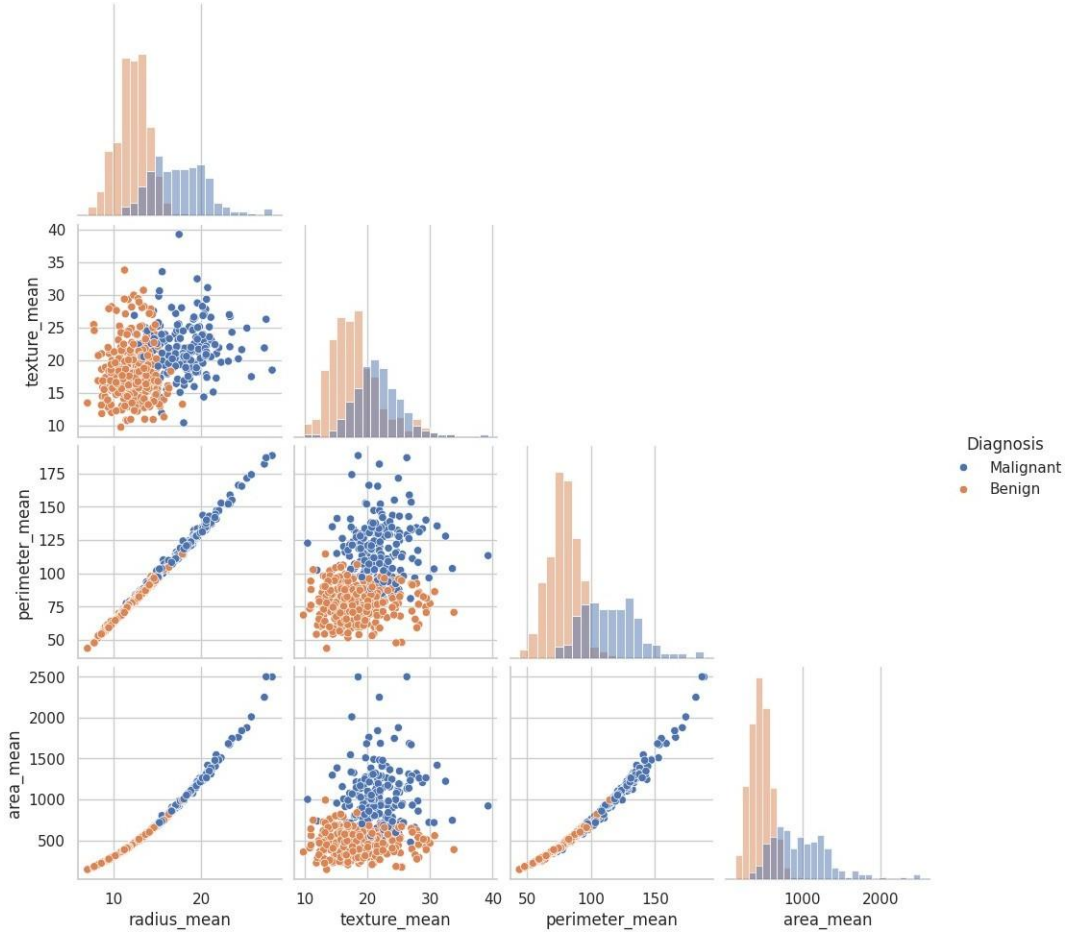


Figure 6: Pairplot of Selected Features by Diagnosis

Key observations: Radius and perimeter lie on an almost straight line, and area follows a smooth curved (quadratic) relationship with both. On the diagonal histograms, the malignant distributions (blue) are shifted to the right of the benign ones (orange). Texture shows the largest overlap between the classes.

Interpretation: The near-perfect relation between radius, perimeter and area confirms the redundancy seen in the heatmap. The shifted histograms confirm that size features are useful for classification, while texture is less discriminative on its own.

Visualization 7: Class-wise Mean Values of All Features (Grouped Bar Chart)

Purpose: To compare the average value of every feature for benign and malignant cases.

What it shows: Grouped bars of the mean of each of the 30 features for the two classes (the underlying numbers were also printed as a table; see Table 4).

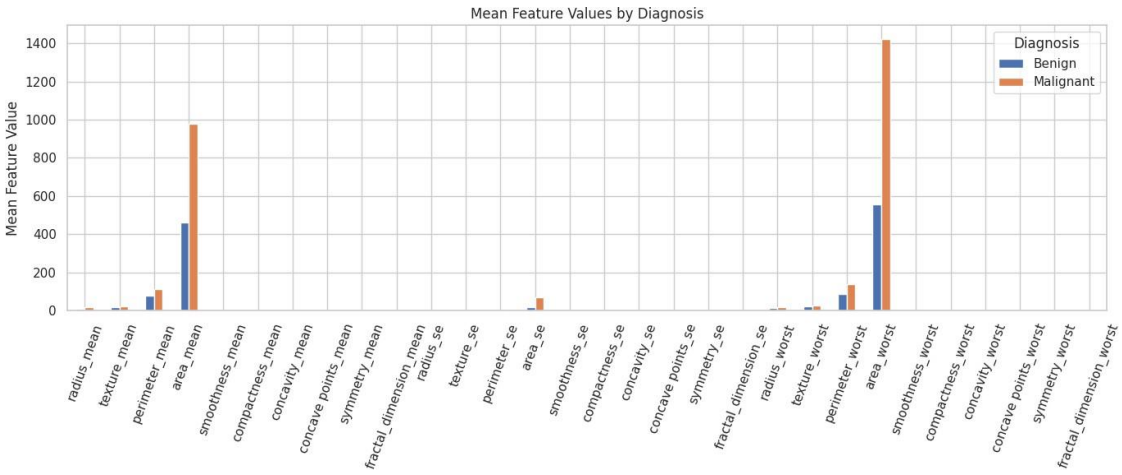


Figure 7: Mean Feature Values by Diagnosis

Key observations: The bars for area_mean, area_worst and perimeter_worst are far taller than the others, and the malignant bar is higher in all of them (for example area_mean: 978.4 for malignant versus 462.8 for benign). All the other bars are almost invisible because those features have much smaller values.

Interpretation: The chart confirms that malignant cases have larger size features, but its readability is limited, because the features are on different scales. The printed table is more informative: for example, concavity_mean (0.161 versus 0.046) and concave points_mean (0.088 versus 0.026) are several times higher in malignant cases, while fractal_dimension_mean is almost identical in the two classes (0.0627 versus 0.0629) and therefore barely helps to separate them.

6. Deep Learning Model

A Multilayer Perceptron (MLP) was chosen because the data are tabular, with 30 numerical inputs and a binary output. An MLP is a feed-forward neural network made of fully connected layers; it can learn non-linear combinations of the input features. The model was implemented with TensorFlow/Keras [5].

6.1 Model Architecture

The selected model has two hidden layers (64 and 32 neurons), dropout after the first hidden layer, and a single sigmoid output neuron that gives the probability that a sample is malignant.

Layer	Type	Output size	Activation	Parameters*
Input	Input layer	30	–	0
Hidden 1	Dense	64	ReLU	1,984
Regularization	Dropout (rate 0.3)	64	–	0
Hidden 2	Dense	32	ReLU	2,080
Output	Dense	1	Sigmoid	33
Total				4,097

*Table 8: MLP architecture. *Parameter counts were calculated from the layer sizes (inputs × neurons + biases); the notebook did not print model.summary().*

- **ReLU (Rectified Linear Unit):** outputs max(0, x). It is simple, fast and helps avoid the vanishing-gradient problem in hidden layers.
- **Sigmoid:** squashes the output to the range 0–1, so it can be interpreted as the probability of the malignant class. A threshold of 0.5 was used to assign the class.
- **Dropout:** randomly switches off 30% of the neurons of the first hidden layer during training, which reduces over-fitting [4].

6.2 Optimizer, Loss Function and Training Parameters

Parameter	Value
Optimizer	Adam, learning rate 0.001 [3]
Loss function	Binary cross-entropy
Metrics tracked	Accuracy and AUC
Batch size	32
Maximum epochs	100
Early stopping	Monitor: validation loss; patience: 12 epochs; restore best weights: True
Random seed	42 (for the data split and for TensorFlow)
Input data	Standardized features (training 398, validation 85 records)
Decision threshold	0.5

Table 9: Training configuration

Early stopping ends training when the validation loss has not improved for 12 consecutive epochs and then restores the weights from the best epoch. The selected model trained for **29 epochs**; therefore, its best epoch was epoch 17 (29 − 12), and the weights from that epoch were used for evaluation.

6.3 Hyperparameter Tuning and Model Selection

A basic tuning step was carried out. Two configurations of the number of hidden neurons and dropout rate were trained under identical conditions. Each was scored using the **AUC on the validation set**, and the one with the higher score was selected. The test set was not used for this selection.

Hidden layers	Dropout	Epochs trained	Validation AUC	Selected
(32, 16)	0.2	39	0.995873	No
(64, 32)	0.3	29	0.997052	Yes

Table 10: Hyperparameter tuning results (actual notebook output)

The (64, 32) network with dropout 0.3 was selected. The difference in validation AUC is very small (about 0.001) and the validation set has only 85 records, so this should be read as “both configurations performed very well and the larger one was marginally better”, not as proof that it is clearly superior. Only two configurations were tried; the learning rate and batch size were fixed.

6.4 Feature Analysis

Feature analysis was carried out with the correlation heatmap (Figure 4), the class-wise comparison (Figure 7) and a permutation-importance analysis (Section 8.6). In permutation importance, one feature at a time is randomly shuffled in the test set, and the drop in AUC shows how much the model relied on it.

7. Result Visualization and Interpretation

7.1 Training and Validation Loss

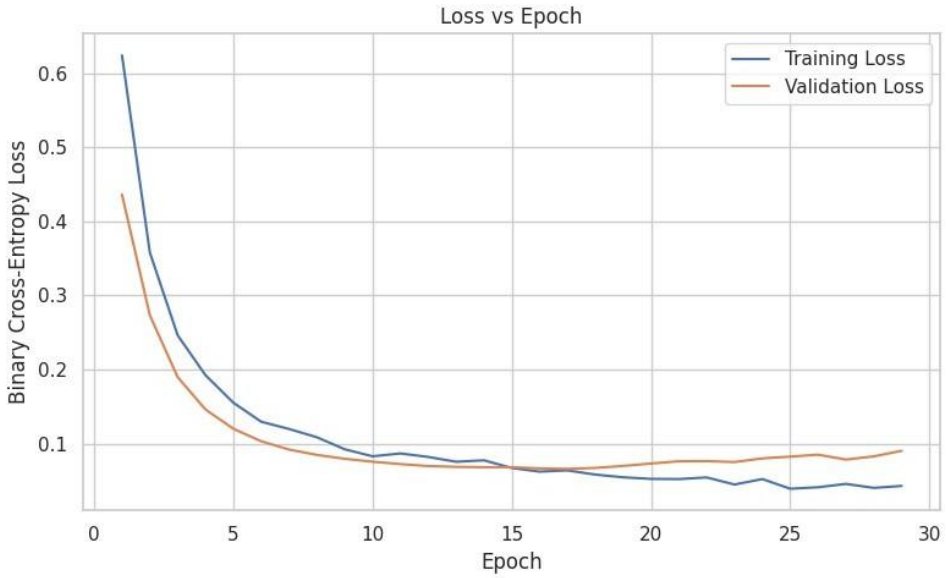


Figure 8: Loss vs Epoch (selected MLP)

What it shows: the binary cross-entropy loss for the training and validation sets at each of the 29 epochs. Lower is better.

Interpretation: both curves fall steeply during the first ~8 epochs (training loss from about 0.62 to about 0.12; validation loss from about 0.44 to about 0.09; values read approximately from the plot). They then remain close to each other, with the validation loss reaching its lowest level at around epochs 15–18. After that, the training loss keeps decreasing slowly while the validation loss rises slightly (to about 0.09 at epoch 29). This is an early sign of over-fitting, and it is the reason why early stopping was used and its best-epoch weights restored. The small gap between the curves indicates that the model generalizes well.

7.2 Training and Validation Accuracy

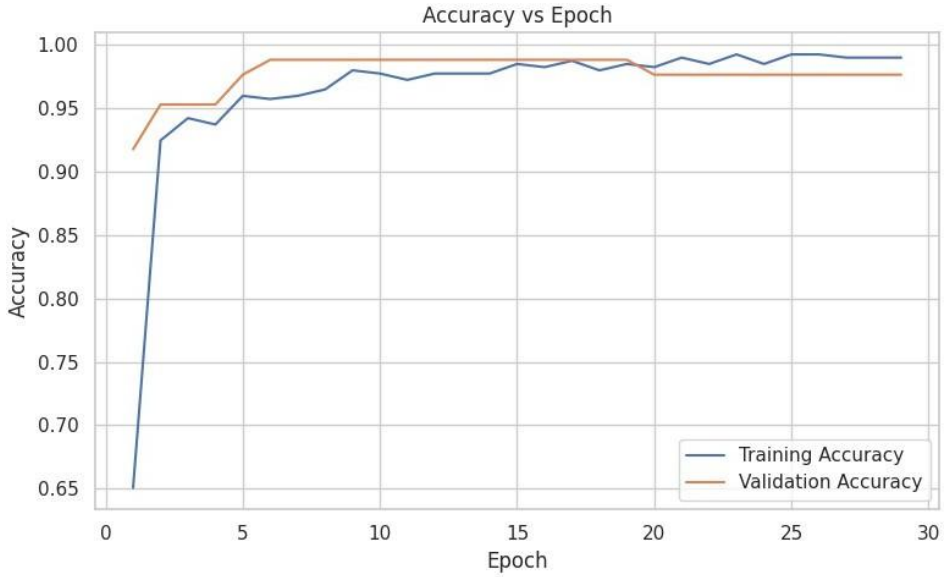


Figure 9: Accuracy vs Epoch (selected MLP)

What it shows: the proportion of correctly classified samples in the training and validation sets at each epoch.

Interpretation: training accuracy rises from about 65% in epoch 1 to about 99% by the end. Validation accuracy is already above 90% in the first epoch and reaches about 98.8% by epoch 6, remaining there until a small dip to about 97.6% at epoch 20. Since the validation set has 85 samples, one sample is about 1.2 percentage points, so these small changes correspond to a single record. The validation accuracy is higher than the training accuracy in the early epochs because dropout is active only during training and the training value is averaged over the epoch.

7.3 Test-Set Performance Metrics

The selected model was evaluated once on the 86 unseen test records. All values below are from the notebook output.

Metric	Value	Meaning
Accuracy	0.9884 (98.84%)	85 of 86 test records classified correctly
Precision (malignant)	1.0000	Of all records predicted malignant, 100% really were malignant
Recall (malignant)	0.9688	Of all truly malignant records, 96.88% were found
F1-score (malignant)	0.9841	Harmonic mean of precision and recall
ROC-AUC	1.0000	Perfect ranking of malignant above benign on this test set

Table 11: Test-set results of the MLP (actual notebook output)

Class	Precision	Recall	F1-score	Support
Benign	0.98	1.00	0.99	54
Malignant	1.00	0.97	0.98	32
Accuracy			0.99	86

Class	Precision	Recall	F1-score	Support
Macro average	0.99	0.98	0.99	86
Weighted average	0.99	0.99	0.99	86

Table 12: Classification report (actual notebook output, rounded to two decimals)

7.4 Confusion Matrix

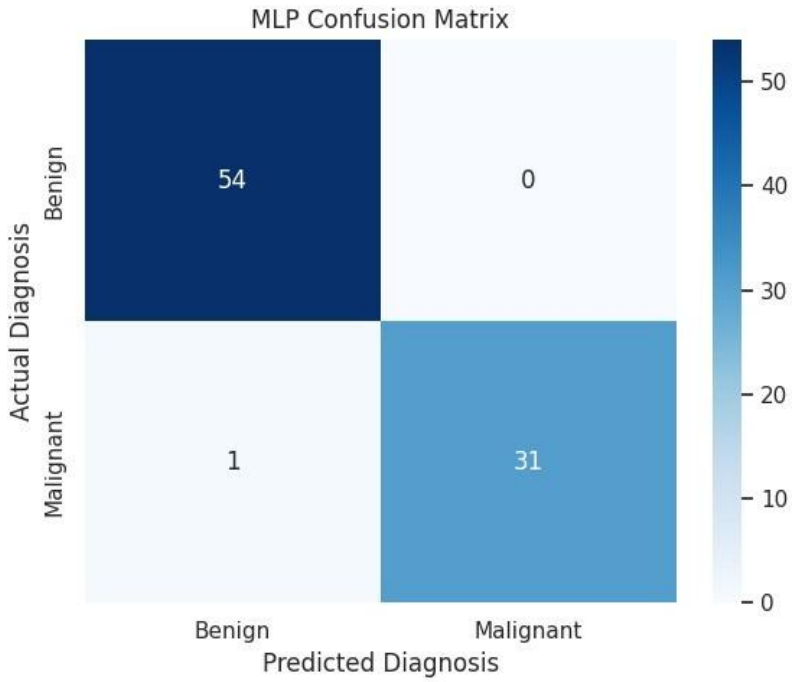


Figure 10: Confusion Matrix of the MLP on the Test Set

What it shows: how many test records of each actual class were assigned to each predicted class. Rows are actual diagnoses; columns are predictions.

	Predicted Benign	Predicted Malignant
Actual Benign (54)	54 (true negatives)	0 (false positives)
Actual Malignant (32)	1 (false negative)	31 (true positives)

Interpretation: all 54 benign cases were classified correctly, and 31 of 32 malignant cases were found. The model made only one mistake: a malignant case predicted as benign (a false negative). In a medical setting, a false negative is generally the more serious kind of error, because a real cancer could be missed; this is why recall for the malignant class is the metric that deserves the most attention. The precision of 1.00 follows from there being no false positives (31 / (31 + 0)).

7.5 ROC Curve and AUC

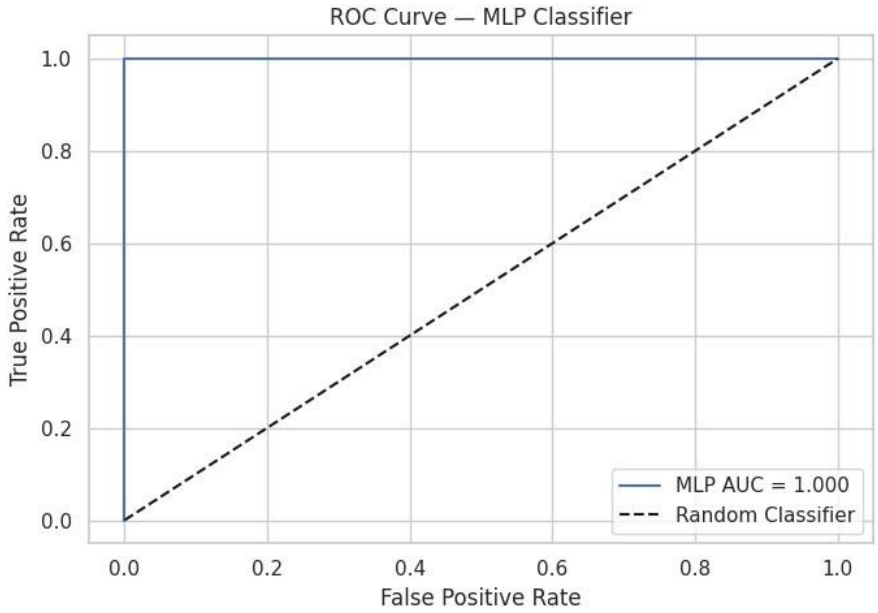


Figure 11: ROC Curve of the MLP Classifier (AUC = 1.000)

What it shows: the trade-off between the true-positive rate (sensitivity) and the false-positive rate as the decision threshold changes. The dashed diagonal is a random classifier (AUC = 0.5); the area under the curve (AUC) summarizes the performance of the model in one number [7].

Interpretation: the curve goes straight up to the top-left corner, giving AUC = 1.0000 (rounded to four decimals). This means that, on this test set, every malignant record received a higher predicted probability than every benign record; so a threshold exists that would have separated all 86 records correctly. At the default threshold of 0.5, one malignant record is still missed, so the choice of threshold matters. If a lower threshold is to be used to improve recall, it should be chosen using the validation set, not the test set. An AUC of 1.0 on only 86 records should not be taken to mean that the model is perfect on new data (see Section 8.7).

7.6 Feature Importance (Permutation Importance)

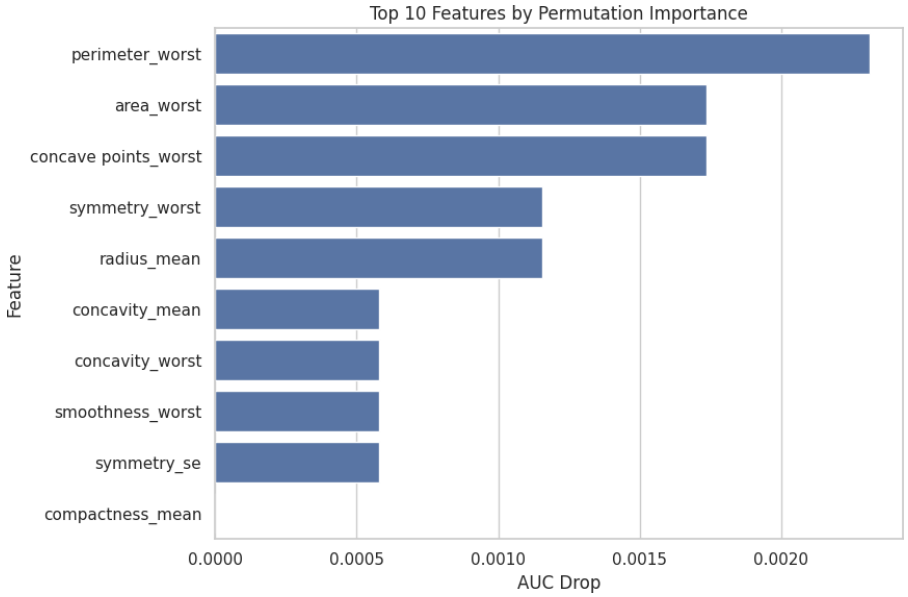


Figure 12: Top 10 Features by Permutation Importance (drop in test AUC)

What it shows: the decrease in test AUC after randomly shuffling one feature. A larger drop means that the model relied more on that feature.

Rank	Feature	AUC drop
1	perimeter_worst	0.002315
2	area_worst	0.001736
3	concave points_worst	0.001736
4	symmetry_worst	0.001157
5	radius_mean	0.001157

Table 13: Top five features by permutation importance (actual notebook output)

Interpretation: the highest-ranked features are the worst-value size and shape measurements (perimeter, area and concave points), which agrees with the EDA, where these features differed most between benign and malignant cases. However, **all of the AUC drops are extremely small** (the largest is 0.0023), because the model's AUC is already almost perfect and the test set is small. Also, since many features are strongly correlated, shuffling one of them may be compensated by the others, which hides its true importance. These results should therefore be treated as a rough indication, not as a precise ranking.

7.7 Comparison with Logistic Regression and Overall Interpretation

Model	Accuracy	F1-score	ROC-AUC
MLP (deep learning)	0.9884	0.9841	1.0000
Logistic Regression (baseline)	0.9767	0.9688	0.9988

Table 14: MLP vs baseline on the same test set (actual notebook output)

The MLP performed slightly better than the Logistic Regression baseline. The accuracy difference (0.9884 versus 0.9767) corresponds to a single test record (85 versus 84 correct predictions out of 86), so the advantage is not statistically meaningful. The key lesson is that this dataset is highly separable: even a simple linear model performs very well.

Statistical caution (additional calculation, not part of the notebook): the test set has only 86 records, 32 of them malignant. An exact 95% binomial confidence interval for the malignant recall of 31/32 is approximately 0.84 to 1.00, and for the accuracy of 85/86 approximately 0.94 to 1.00. The true performance on new data may therefore be noticeably lower than the single test result. Cross-validation or an external dataset would give a more reliable estimate.

Medical disclaimer: these results come from a small research dataset and an academic experiment. They do not show that the model can diagnose real patients or replace medical professionals.

8. Conclusion and Future Scope

8.1 What Was Learned from the Dataset

The Breast Cancer Wisconsin (Diagnostic) dataset is clean (no missing values or duplicates) and contains 569 records and 30 numerical features describing cell nuclei. The classes are

moderately imbalanced (357 benign, 212 malignant). The features are right-skewed, lie on very different scales, and are highly correlated, especially radius, perimeter and area.

8.2 Key Patterns and Findings

- Malignant cases have clearly larger size features (e.g. mean area 978 vs 463) and larger concavity and concave-points values than benign cases.
- Some features, such as fractal_dimension_mean, hardly differ between the classes.
- No single pair of features separates the classes perfectly (Figure 5), but the combination of all features does so very well.
- Worst-value size and shape features were the most influential features for the MLP, although the importance values were small.

8.3 Model Performance

The selected MLP (two hidden layers of 64 and 32 neurons, dropout 0.3, Adam optimizer) achieved on the 86-record test set an accuracy of 98.84%, precision of 1.0000, recall of 0.9688, F1-score of 0.9841 and ROC-AUC of 1.0000, with one false negative and no false positives. It was slightly better than a Logistic Regression baseline (accuracy 97.67%), but the difference amounts to one record.

8.4 Challenges and Limitations

- **Small dataset and small test set:** with only 86 test records, one mistake changes the accuracy by about 1.2 percentage points, and the metrics have wide confidence intervals.
- **Near-perfect scores:** the AUC of 1.0 can hide limitations; the dataset is known to be easily separable, and results on other hospitals, devices or populations may differ.
- **Multicollinearity:** highly correlated features make it difficult to interpret feature importance.
- **Outliers:** flagged outliers were retained rather than treated, and their effect on the model was not tested separately.
- **Limited tuning:** only two hyperparameter configurations were compared.
- **Visualization limits:** the boxplot and the class-wise bar chart place features with very different scales on one axis, which reduces readability; no interactive visualization and no formal distribution fitting or normality tests were included.
- **Medical use:** the model is not clinically validated and must not be used to diagnose real patients.

8.5 Future Scope (suggestions, not yet performed)

- Use stratified k-fold cross-validation, or repeated splits, to obtain more reliable performance estimates.
- Extend the hyperparameter search (learning rate, batch size, number of layers, regularization) with grid or random search, or a tool such as Keras Tuner.
- Choose the decision threshold with the validation set to prioritise recall of the malignant class, and check probability calibration.
- Reduce redundancy among correlated features (e.g. Principal Component Analysis or feature selection) and compare the results.
- Explain predictions with SHAP or LIME and calculate permutation importance on the validation set with repeated shuffles.

- Add Q–Q plots, normality tests and probability-distribution fitting, class-wise boxplots on standardized features, and interactive Plotly dashboards.
- Validate the approach on external data and, for a real-world application, consult clinical experts; image-based models (CNN) on the original FNA images could also be explored.

9. References

(IEEE style)

[1] UCI Machine Learning, "Breast Cancer Wisconsin (Diagnostic) Data Set," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data.

[2] W. N. Street, W. H. Wolberg, and O. L. Mangasarian, "Nuclear feature extraction for breast tumor diagnosis," in Proc. IS&T/SPIE Int. Symp. Electronic Imaging: Science and Technology, vol. 1905, 1993, pp. 861–870.

[3] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. 3rd Int. Conf. Learning Representations (ICLR), 2015. [Online]. Available: arXiv:1412.6980.

[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," J. Mach. Learn. Res., vol. 15, no. 56, pp. 1929–1958, 2014.

[5] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2016, pp. 265–283.

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[7] T. Fawcett, "An introduction to ROC analysis," Pattern Recognit. Lett., vol. 27, no. 8, pp. 861–874, 2006.

[8] B. Fry, "Computational information design," Ph.D. dissertation, Program in Media Arts and Sciences, Massachusetts Institute of Technology, Cambridge, MA, USA, 2004.

[9] J. D. Hunter, "Matplotlib: A 2D graphics environment," Comput. Sci. Eng., vol. 9, no. 3, pp. 90–95, 2007.

[10] M. L. Waskom, "seaborn: statistical data visualization," J. Open Source Softw., vol. 6, no. 60, p. 3021, 2021.

10. Appendix (Code Section)

A.1 How the Code Is Organized

The complete Python code is in a single Google Colab notebook, EDA_Breast_Cancer_Assignment.ipynb (34 cells: 18 code cells and 16 short markdown headings), which runs top to bottom. The code was written for Python 3 with pandas, NumPy, Matplotlib, Seaborn, scikit-learn and TensorFlow/Keras [5], [6], [9], [10]. It expects the Kaggle file data.csv at /content/data.csv. Table 15 maps the notebook cells to the sections of this report.

Cells	Content	Report section
0	Imports; data loading; shape, head, info and describe	5.1, 4
1	Drop id and Unnamed: 32; duplicates; missing values; label encoding (B→0, M→1)	5.2–5.5
3	Class distribution count plot	6 (Viz. 1)
5	Histograms and skewness	6 (Viz. 2)
7	Boxplots and IQR outlier counts	5.4, 6 (Viz. 3)
9	Correlation heatmap	6 (Viz. 4)
11	Scatter plot radius vs texture	6 (Viz. 5)
13	Pairplot	6 (Viz. 6)
15	Class-wise mean comparison and bar chart	4.4, 6 (Viz. 7)
17	Stratified 70/15/15 split; median imputer; standard scaler	5.6, 5.7
19–20	Install TensorFlow; build MLP; train two configurations; select by validation AUC	7
22	Loss and accuracy curves	8.1, 8.2
24	Test accuracy, precision, recall, F1, ROC-AUC, classification report	8.3
26	Confusion matrix	8.4
28	ROC curve	8.5
30	Permutation feature importance	8.6
32	Logistic Regression baseline comparison	8.7
33	Saves the cleaned data, result tables, model and preprocessing objects	–

Table 15: Mapping of notebook cells (numbered from 0) to report sections

A.2 Notebook Code

The code below is copied from the notebook without changes (outputs omitted). Cell numbers refer to the cells of the notebook; markdown cells are shown as comments.

```
# ---- Cell 0 (code) ----
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
import seaborn as sns

# Path to the dataset
```

```
csv_path = "/content/data.csv"

if not os.path.exists(csv_path):
    raise FileNotFoundError(f"Could not find the dataset at {csv_path}")

df = pd.read_csv(csv_path)

print("Dataset shape:", df.shape)
print("\nFirst five rows:")
display(df.head())

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types and non-null counts:")
df.info()

print("\nStatistical summary:")
display(df.describe().T)
```

```
# ---- Cell 1 (code) ----

# Remove unnecessary columns if present
df = df.drop(
    columns=["id", "Unnamed: 32"],
    errors="ignore"
)

# Remove duplicate rows
print("Duplicate rows:", df.duplicated().sum())
df = df.drop_duplicates().copy()

# Check missing values
print("\nMissing values per column:")
print(df.isnull().sum().sort_values(ascending=False).head(10))

# Check target values
print("\nDiagnosis counts:")
print(df["diagnosis"].value_counts(dropna=False))

# Separate features and target
df = df.dropna(subset=["diagnosis"]).copy()

X = df.drop(columns=["diagnosis"]).apply(
    pd.to_numeric, errors="coerce"
)

y = df["diagnosis"].map({"B": 0, "M": 1})

# Remove rows with invalid target labels, if any
valid = y.notna()
X = X.loc[valid].copy()
y = y.loc[valid].astype(int)

# Report missing feature values before imputation
print("\nMissing feature values:", X.isnull().sum().sum())

print("\nCleaned feature shape:", X.shape)
print("Target shape:", y.shape)
print("\nClass labels: 0 = Benign, 1 = Malignant")
```

```
# [Cell 2] Exploratory Data Analysis

# ---- Cell 3 (code) ----
```

```
sns.set_theme(style="whitegrid")

plt.figure(figsize=(7, 5))
ax = sns.countplot(
    x=y.map({0: "Benign", 1: "Malignant"}),
    hue=y.map({0: "Benign", 1: "Malignant"}),
    legend=False
)
ax.set_title("Distribution of Breast Cancer Diagnoses")
ax.set_xlabel("Diagnosis")
ax.set_ylabel("Number of Cases")
plt.tight_layout()
plt.show()
# [Cell 4] Histogram-Univariate analysis
# ---- Cell 5 (code) ----

selected_features = [
    "radius_mean",
    "texture_mean",
    "perimeter_mean",
    "area_mean"
]

available_features = [
    col for col in selected_features if col in X.columns
]

X[available_features].hist(
    figsize=(12, 8),
    bins=20,
    edgecolor="black"
)

plt.suptitle("Distributions of Selected Numerical Features")
plt.tight_layout()
plt.show()

print("Feature skewness:")
display(X[available_features].skew().to_frame("Skewness"))
# [Cell 6] Boxplots-Outlier analysis
# ---- Cell 7 (code) ----

plt.figure(figsize=(12, 6))

sns.boxplot(data=X[available_features], orient="h")

plt.title("Boxplots of Selected Numerical Features")
plt.xlabel("Feature Value")
plt.tight_layout()
plt.show()

# IQR-based outlier counts
for col in X.columns:
    q1 = X[col].quantile(0.25)
    q3 = X[col].quantile(0.75)
    iqr = q3 - q1

    count = (
        (X[col] < q1 - 1.5 * iqr) |
        (X[col] > q3 + 1.5 * iqr)
    ).sum()
```

```
    if count > 0:
        print(f"{col}: {count} potential outliers")
# [Cell 8] Correlation heatmap
# ---- Cell 9 (code) ----

plt.figure(figsize=(15, 11))

correlation = X.corr()

sns.heatmap(
    correlation,
    cmap="coolwarm",
    center=0,
    linewidths=0.1
)

plt.title("Correlation Heatmap of Numerical Features")
plt.tight_layout()
plt.show()
# [Cell 10] Scatterplot-Feature relationships
# ---- Cell 11 (code) ----

plot_df = X.copy()
plot_df["Diagnosis"] = y.map({
    0: "Benign",
    1: "Malignant"
})

plt.figure(figsize=(8, 6))

sns.scatterplot(
    data=plot_df,
    x="radius_mean",
    y="texture_mean",
    hue="Diagnosis",
    alpha=0.75
)

plt.title("Mean Radius vs Mean Texture by Diagnosis")
plt.tight_layout()
plt.show()
# [Cell 12] Pairplot-Multivariate analysis
# ---- Cell 13 (code) ----

pair_features = [
    col for col in [
        "radius_mean",
        "texture_mean",
        "perimeter_mean",
        "area_mean"
    ]
    if col in X.columns
]

pair_df = X[pair_features].copy()
pair_df["Diagnosis"] = y.map({
    0: "Benign",
    1: "Malignant"
})

sns.pairplot(
```

```
    pair_df,
    hue="Diagnosis",
    diag_kind="hist",
    corner=True
)

plt.show()
# [Cell 14] Compare feature means by diagnosis
# ---- Cell 15 (code) ----

mean_comparison = X.assign(
    Diagnosis=y.map({0: "Benign", 1: "Malignant"})
).groupby("Diagnosis").mean().T

display(mean_comparison)

mean_comparison.plot(
    kind="bar",
    figsize=(14, 6)
)

plt.title("Mean Feature Values by Diagnosis")
plt.ylabel("Mean Feature Value")
plt.xticks(rotation=70)
plt.tight_layout()
plt.show()
# [Cell 16] Split the data and prepare the features
# ---- Cell 17 (code) ----

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

# 70% training, 30% temporary
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

# Split the remaining 30% equally
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=0.50,
    random_state=42,
    stratify=y_temp
)

# Learn imputation values from training data only
imputer = SimpleImputer(strategy="median")

X_train_imp = imputer.fit_transform(X_train)
X_val_imp = imputer.transform(X_val)
X_test_imp = imputer.transform(X_test)

# Learn scaling parameters from training data only
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train_imp)
X_val_scaled = scaler.transform(X_val_imp)
X_test_scaled = scaler.transform(X_test_imp)
```

```
print("Training set:", X_train_scaled.shape)
print("Validation set:", X_val_scaled.shape)
print("Test set:", X_test_scaled.shape)
# [Cell 18] Train the deep learning model
# ---- Cell 19 (code) ----

%pip -q install tensorflow
# ---- Cell 20 (code) ----

import tensorflow as tf

from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import roc_auc_score

tf.keras.utils.set_random_seed(42)

def build_model(hidden_units=(32, 16), dropout_rate=0.2):
    model = Sequential([
        Input(shape=(X_train_scaled.shape[1],)),
        Dense(hidden_units[0], activation="relu"),
        Dropout(dropout_rate),
        Dense(hidden_units[1], activation="relu"),
        Dense(1, activation="sigmoid")
    ])

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss="binary_crossentropy",
        metrics=["accuracy", tf.keras.metrics.AUC(name="auc")]
    )

    return model

configurations = [
    {"units": (32, 16), "dropout": 0.2},
    {"units": (64, 32), "dropout": 0.3}
]

results = []
trained_models = []

for config in configurations:
    tf.keras.backend.clear_session()
    tf.keras.utils.set_random_seed(42)

    model = build_model(
        hidden_units=config["units"],
        dropout_rate=config["dropout"]
    )

    early_stop = EarlyStopping(
        monitor="val_loss",
        patience=12,
        restore_best_weights=True
    )

    history = model.fit(
        X_train_scaled,
        y_train,
```

```
        validation_data=(X_val_scaled, y_val),
        epochs=100,
        batch_size=32,
        callbacks=[early_stop],
        verbose=0
    )

    val_prob = model.predict(
        X_val_scaled, verbose=0
    ).ravel()

    val_auc = roc_auc_score(y_val, val_prob)

    results.append({
        "Hidden layers": str(config["units"]),
        "Dropout": config["dropout"],
        "Epochs trained": len(history.history["loss"]),
        "Validation AUC": val_auc
    })

    trained_models.append((model, history))

tuning_results = pd.DataFrame(results)
display(tuning_results)

best_index = tuning_results["Validation AUC"].idxmax()
best_model, best_history = trained_models[best_index]

print("Selected configuration:")
print(tuning_results.iloc[best_index])
# [Cell 21] Plot training and validation curves
# ---- Cell 22 (code) ----

history = best_history.history
epochs = range(1, len(history["loss"]) + 1)

plt.figure(figsize=(8, 5))
plt.plot(epochs, history["loss"], label="Training Loss")
plt.plot(epochs, history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("Loss vs Epoch")
plt.legend()
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(epochs, history["accuracy"], label="Training Accuracy")
plt.plot(epochs, history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Accuracy vs Epoch")
plt.legend()
plt.tight_layout()
plt.show()
# [Cell 23] Evaluate the model on the test set
# ---- Cell 24 (code) ----

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
```

```
    f1_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    roc_auc_score
)

test_prob = best_model.predict(
    X_test_scaled, verbose=0
).ravel()

test_pred = (test_prob >= 0.5).astype(int)

print("Test Accuracy:", round(
    accuracy_score(y_test, test_pred), 4
))
print("Precision:", round(
    precision_score(y_test, test_pred, zero_division=0), 4
))
print("Recall:", round(
    recall_score(y_test, test_pred, zero_division=0), 4
))
print("F1 Score:", round(
    f1_score(y_test, test_pred, zero_division=0), 4
))
print("ROC-AUC:", round(
    roc_auc_score(y_test, test_prob), 4
))

print("\nClassification Report:")
print(classification_report(
    y_test,
    test_pred,
    target_names=["Benign", "Malignant"],
    zero_division=0
))
# [Cell 25] Confusion Matrix
# ---- Cell 26 (code) ----

cm = confusion_matrix(y_test, test_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Benign", "Malignant"],
    yticklabels=["Benign", "Malignant"]
)

plt.xlabel("Predicted Diagnosis")
plt.ylabel("Actual Diagnosis")
plt.title("MLP Confusion Matrix")
plt.tight_layout()
plt.show()
# [Cell 27] ROC Curve
# ---- Cell 28 (code) ----

fpr, tpr, thresholds = roc_curve(y_test, test_prob)
test_auc = roc_auc_score(y_test, test_prob)
```

```
plt.figure(figsize=(7, 5))
plt.plot(fpr, tpr, label=f"MLP AUC = {test_auc:.3f}")
plt.plot([0, 1], [0, 1], "k--", label="Random Classifier")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve — MLP Classifier")
plt.legend()
plt.tight_layout()
plt.show()
# [Cell 29] Feature importance
# ---- Cell 30 (code) ----

baseline_auc = roc_auc_score(y_test, test_prob)

rng = np.random.default_rng(42)
importance_scores = []

for i, feature in enumerate(X.columns):
    shuffled = X_test_scaled.copy()
    shuffled[:, i] = rng.permutation(shuffled[:, i])

    shuffled_prob = best_model.predict(
        shuffled, verbose=0
    ).ravel()

    shuffled_auc = roc_auc_score(y_test, shuffled_prob)

    importance_scores.append({
        "Feature": feature,
        "AUC Drop": baseline_auc - shuffled_auc
    })

importance_df = pd.DataFrame(importance_scores)
importance_df = importance_df.sort_values(
    "AUC Drop", ascending=False
)

display(importance_df.head(10))

plt.figure(figsize=(9, 6))
sns.barplot(
    data=importance_df.head(10),
    x="AUC Drop",
    y="Feature"
)

plt.title("Top 10 Features by Permutation Importance")
plt.tight_layout()
plt.show()
# [Cell 31] Compare with Logistic Regression
# ---- Cell 32 (code) ----

from sklearn.linear_model import LogisticRegression

baseline_model = LogisticRegression(max_iter=2000)
baseline_model.fit(X_train_scaled, y_train)

lr_pred = baseline_model.predict(X_test_scaled)
lr_prob = baseline_model.predict_proba(X_test_scaled)[:, 1]

comparison = pd.DataFrame([
```

```
    {
        "Model": "MLP Deep Learning",
        "Accuracy": accuracy_score(y_test, test_pred),
        "F1 Score": f1_score(y_test, test_pred),
        "ROC-AUC": roc_auc_score(y_test, test_prob)
    },
    {
        "Model": "Logistic Regression",
        "Accuracy": accuracy_score(y_test, lr_pred),
        "F1 Score": f1_score(y_test, lr_pred),
        "ROC-AUC": roc_auc_score(y_test, lr_prob)
    }
])

display(comparison.round(4))
# ---- Cell 33 (code) ----

import os
import shutil

os.makedirs("figures", exist_ok=True)
os.makedirs("results", exist_ok=True)
os.makedirs("models", exist_ok=True)

# Save cleaned dataset
df.to_csv("results/cleaned_breast_cancer.csv", index=False)

# Save model results
tuning_results.to_csv("results/hyperparameter_tuning.csv", index=False)
comparison.to_csv("results/model_comparison.csv", index=False)
importance_df.to_csv("results/feature_importance.csv", index=False)

# Save model and preprocessing objects
best_model.save("models/best_mlp_model.keras")

import joblib
joblib.dump(imputer, "models/imputer.pkl")
joblib.dump(scaler, "models/scaler.pkl")

print("Project outputs saved successfully.")
```